DA5400 (July-Nov 2025)
Spam–Ham Classification Using Machine Learning
Assignment 3 Report
Nishchala Mukku (EE24S004)

1 Aim

To build a spam-ham classifier for emails using supervised techniques (classification algorithms).

2 Dataset Description

2.1 Training dataset

I have used the Enron dataset for this task. The dataset has a large amount of emails which have been classified as spam and ham. There are about 2004 Spam emails and 2035 Ham emails in the training dataset. For validating our models, 200 Spam and 201 Ham emails are used from this dataset. This can be seen in fig 1.

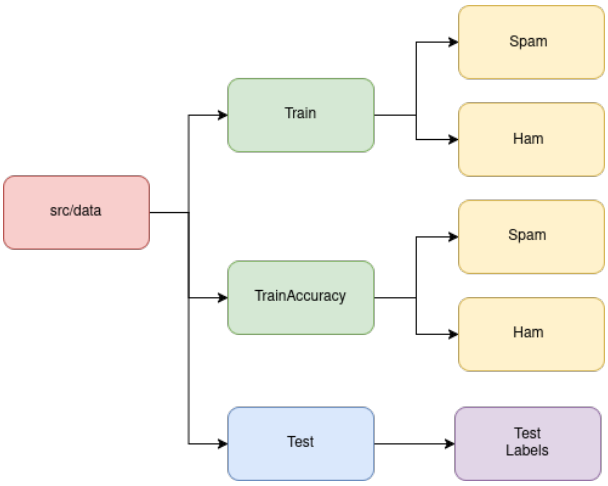

Figure 1: Dataset split

2.2 Test Data Generation

To evaluate the final classifier, a synthetic test set of 100 emails was created using a Python script. The script generates **50 ham** and **50 spam** messages using small sets of hand-crafted templates. Ham templates resemble workplace or conversational emails (meeting reminders, document sharing, project updates), while spam templates contain typical phishing or promotional patterns (prize scams, account warnings, discount offers).

For each message, placeholders such as `{name}`, `{event}`, `{url}`, and `{product}` are filled with randomly sampled values from predefined lists. The 100 generated messages are then randomly shuffled and saved as

`email1.txt`, `email2.txt`, . . . , `email100.txt`

inside the directory `src/data/test/`. A corresponding file `test_labels.csv` is also created, containing the ground-truth labels in the format:

$$\texttt{filename,label} \quad (0 = \text{ham},\ +1 = \text{spam})$$

3 Program flow

The parent directory **src** contains all the necessary python scripts and data. The distribution is shown in fig 2. The pipeline of scripts can be clearly understood from the flowchart in fig 3.

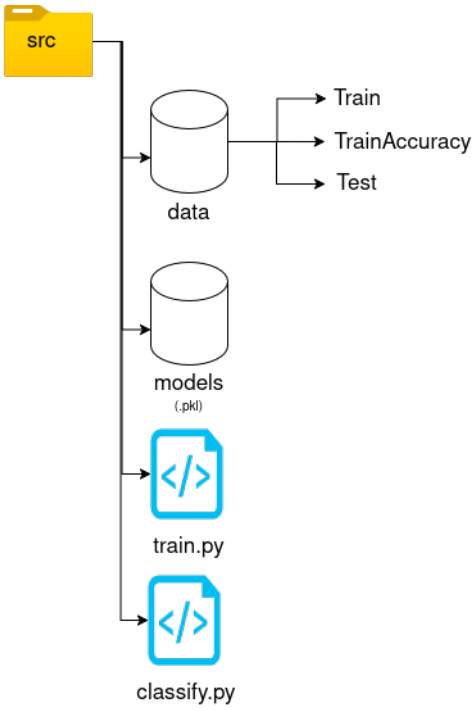

Figure 2: Directory map

4 Data Preprocessing

The preprocessing pipeline was designed to be simple, deterministic, and implementable from scratch.

4.1 Text Cleaning

Each email undergoes:

1. Lowercasing all alphabetic characters.
2. Tokenisation using a whitespace and punctuation based parser.
3. Stopword removal using a custom stopword list derived from NLTK.

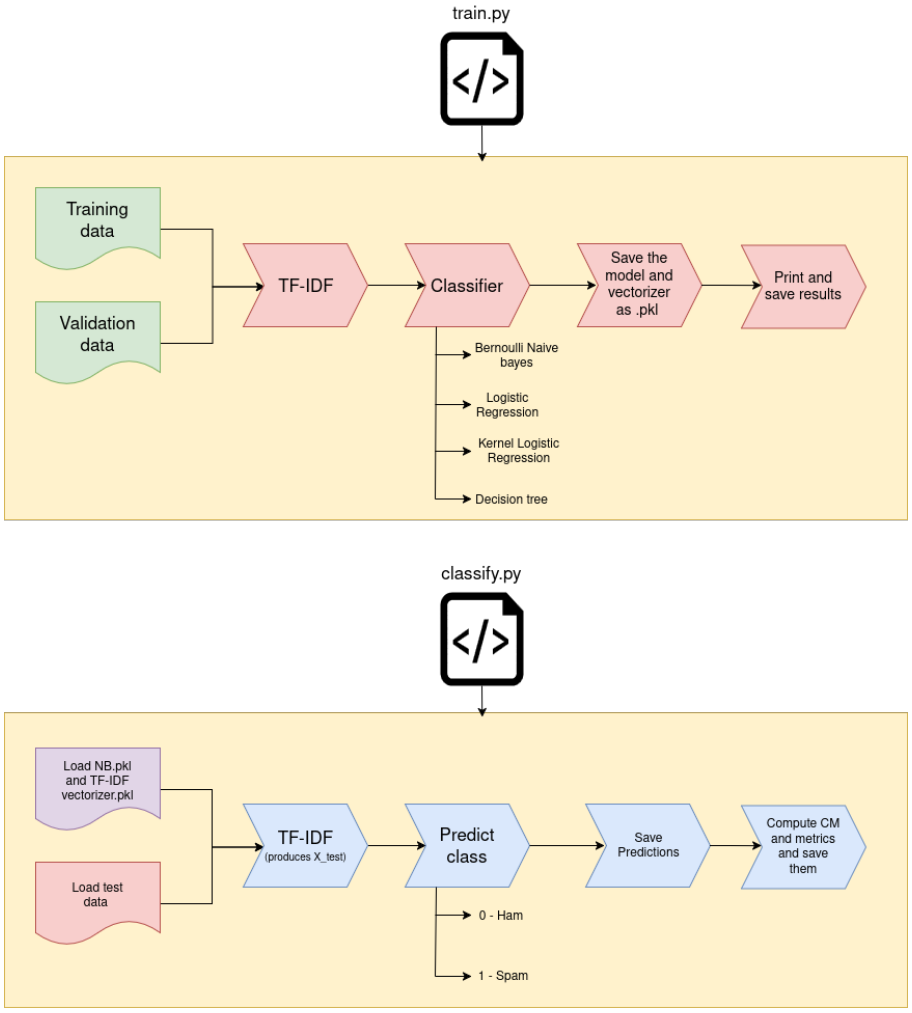

Figure 3: Pipeline of train.py and classify.py

4. Removal of empty tokens.

The goal is to remove uninformative words ("and", "the", "is") and retain signal-bearing terms such as "free", "award", "urgent", "money", which are strongly indicative of spam.

4.2 Vocabulary Construction

From the processed training corpus, all terms with document frequency:

$$\texttt{min\_df} = 2, \qquad \texttt{max\_df} = 0.95$$

were retained. This creates a stable vocabulary while eliminating:

- Extremely rare terms (noise)
- Extremely common terms (uninformative)

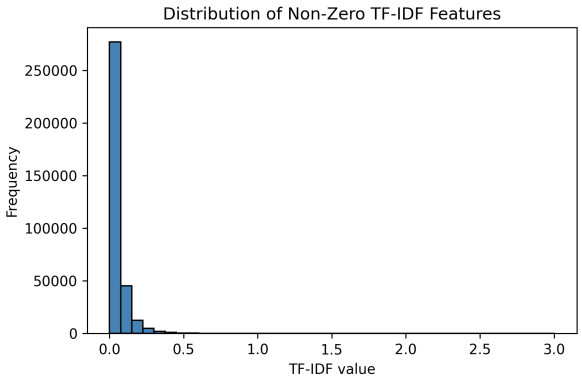

Figure 4: TF-IDF Features distribution

5 Feature Extraction

5.1 TF–IDF Vectorisation

A Term–Frequency Inverse–Document–Frequency (TF–IDF) vectoriser was implemented from scratch. For each document d and term t, the TF–IDF value is:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log\left(\frac{N}{\text{df}(t)}\right),$$

where:

$$\text{tf}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total tokens in } d}, \qquad \text{df}(t) = |\{d : t \in d\}|, \qquad N = \text{number of documents.}$$

TF–IDF is particularly well suited for spam detection because:

- Spam messages contain distinctive high-value keywords ("win", "free", "claim", "urgent")
- Ham messages typically include conversational words with low IDF

Histograms of TF-IDF values across the dataset were generated to visualize class-separating behavior in fig 4.

6 Models Implemented and results

The assignment requires that all algorithms except SVM must be coded from scratch. Accordingly, the following models were implemented without using machine learning libraries.

6.1 Bernoulli Naive Bayes

Bernoulli NB models each term as a binary indicator:

$$p(x_i = 1 \mid y = c) = \theta_{ic}.$$

The full likelihood is:

$$p(x \mid y = c) = \prod_{i=1}^{V} \theta_{ic}^{x_i} (1 - \theta_{ic})^{1 - x_i}.$$

Laplace smoothing with $\alpha = 10^{-2}$ was applied. This model performed the best among all handwritten implementations due to the binary nature of TF–IDF (presence or absence of keyword-like tokens).

Results: The confusion matrix produced on the validation dataset is as in fig 5. We get an accuracy **99%**.

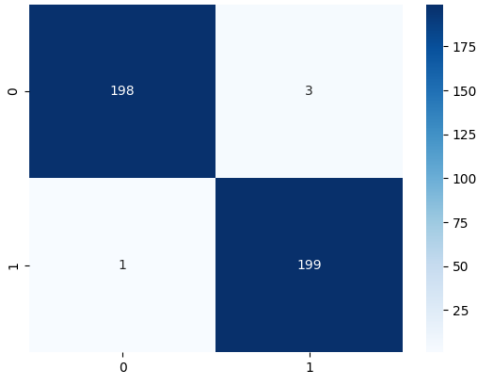

Figure 5: Confusion matrix for Naive bayes

6.2 Logistic Regression (Linear)

A from-scratch implementation was created using gradient descent on the loss:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} \log\left(1 + \exp(-y_i(w^\top x_i + b))\right).$$

Updates:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

Results: The confusion matrix produced on the validation dataset is as in fig 6. We get an accuracy **50.87%**.

6.3 Kernel Logistic Regression (Nonlinear)

To model nonlinear spam–ham boundaries, the dual formulation was implemented:

$$f(x) = \sum_{i=1}^{N} \alpha_i K(x, x_i),$$

$$p(y = 1 \mid x) = \sigma(f(x)), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

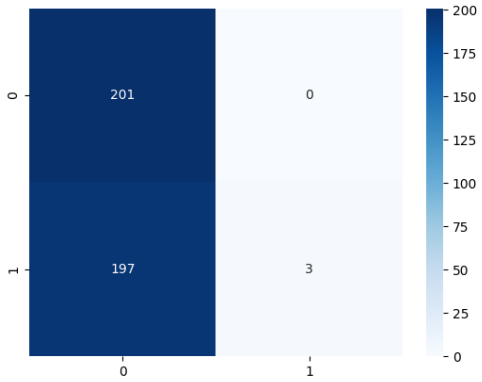

Figure 6: Confusion matrix for Logistic regression

Using an RBF kernel:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2).$$

Gradient descent was performed directly on the dual coefficients α:

$$\alpha \leftarrow \alpha - \eta \nabla_\alpha L.$$

Results: The confusion matrix produced on the validation dataset is as in fig 7. We get an accuracy **49.88%**.

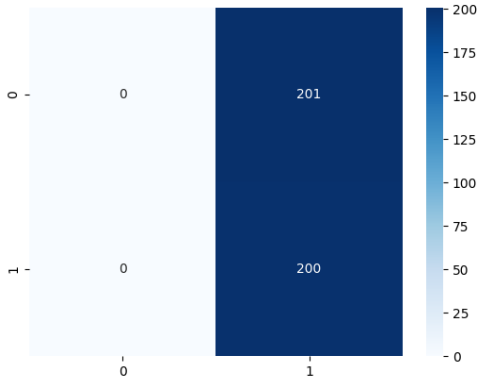

Figure 7: Confusion matrix for Kernel (RBF) Logistic regression

6.4 Decision Tree

A decision tree classifier was implemented from scratch using the **Entropy** impurity measure and **Information Gain** as the splitting criterion. Unlike Gini-based trees, the entropy formulation explicitly measures the uncertainty in the class distribution of a node:

$$H(Y) = -\sum_{c \in \{0,1\}} p(c) \log_2 p(c),$$

where $p(c)$ is the proportion of samples belonging to class c in the node.

For a candidate split on feature j at threshold t, the information gain is:

$$IG(j,t) = H(Y) - \left(\frac{N_L}{N} H(Y_L) + \frac{N_R}{N} H(Y_R) \right),$$

where Y_L and Y_R denote the class labels in the left and right child nodes, and N_L, N_R their respective sizes.

Results: The confusion matrix produced on the validation dataset is as in fig 8. We get an accuracy **68.58%**.

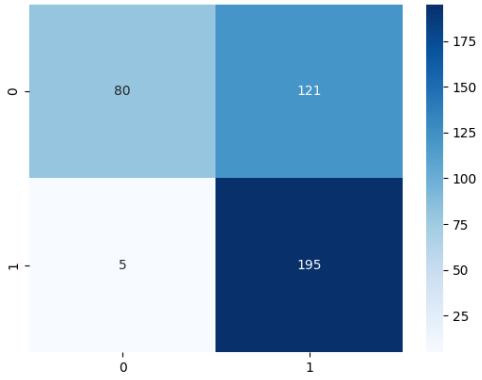

Figure 8: Confusion matrix for Kernel (RBF) Logistic regression

7 Hyperparameter Tuning

Grid-style tuning was performed for:

- Logistic regression learning rate and epochs
- Kernel logistic regression: γ, learning rate, λ
- Decision tree: depth and max features

The best performing hyperparameters were selected based on validation accuracy.

8 Evaluation and Metrics

The validation set was used to evaluate each model. The following quantitative metrics were computed:

- Accuracy
- Confusion matrix (fig 5, 6, 7, 8)
- ROC curve (for LR(fig 9) and KLR(fig 10))

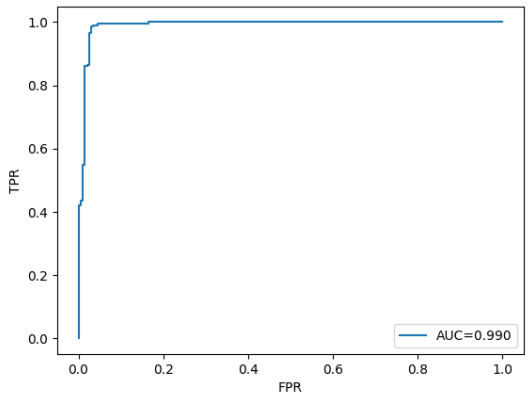

Figure 9: ROC for Logistic regression

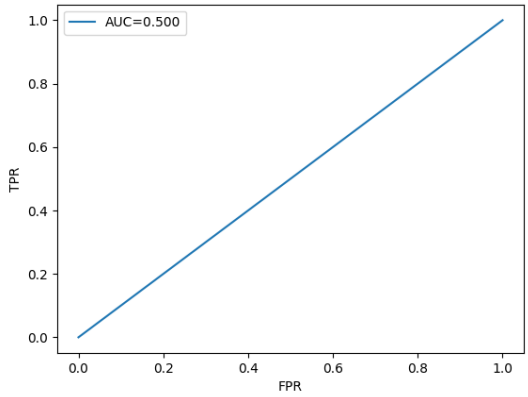

Figure 10: ROC for Kernel Logistic regression

9 Final Model Selection

Although multiple models were implemented, the results showed that:

- Bernoulli Naive Bayes achieved the highest validation accuracy.
- Logistic Regression gave interpretable linear behaviour
- Kernel Logistic Regression captured nonlinear patterns but suffered numerical instability due to the high-dimensional TF–IDF space
- Decision Tree provided feature-based interpretability

Based on performance and computational cost, Bernoulli NB was chosen as the final classifier. This classifier is used by the `classify.py` script to read the emails in the `test` folder and output +1 or 0 predictions.The confusion matrix produced on the test data is as in fig 11. The metrics on the test data are as in given 1.

Table 1: Test set classification metrics (Naive Bayes)

Metric	Value
Accuracy	0.9300
Precision	1.0000
Recall	0.8600
F1-score	0.9247

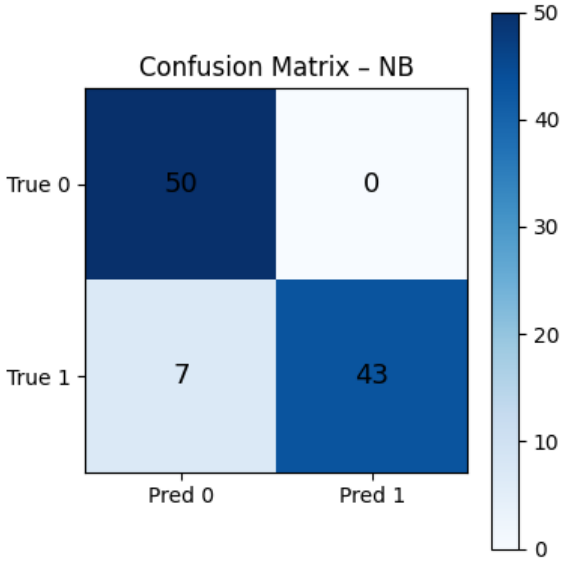

Figure 11: Confusion matrix on test data